

Dossier de projet – LineUp

Une solution digitale pour mieux attendre

Candidat : Elyes MESSAADIA

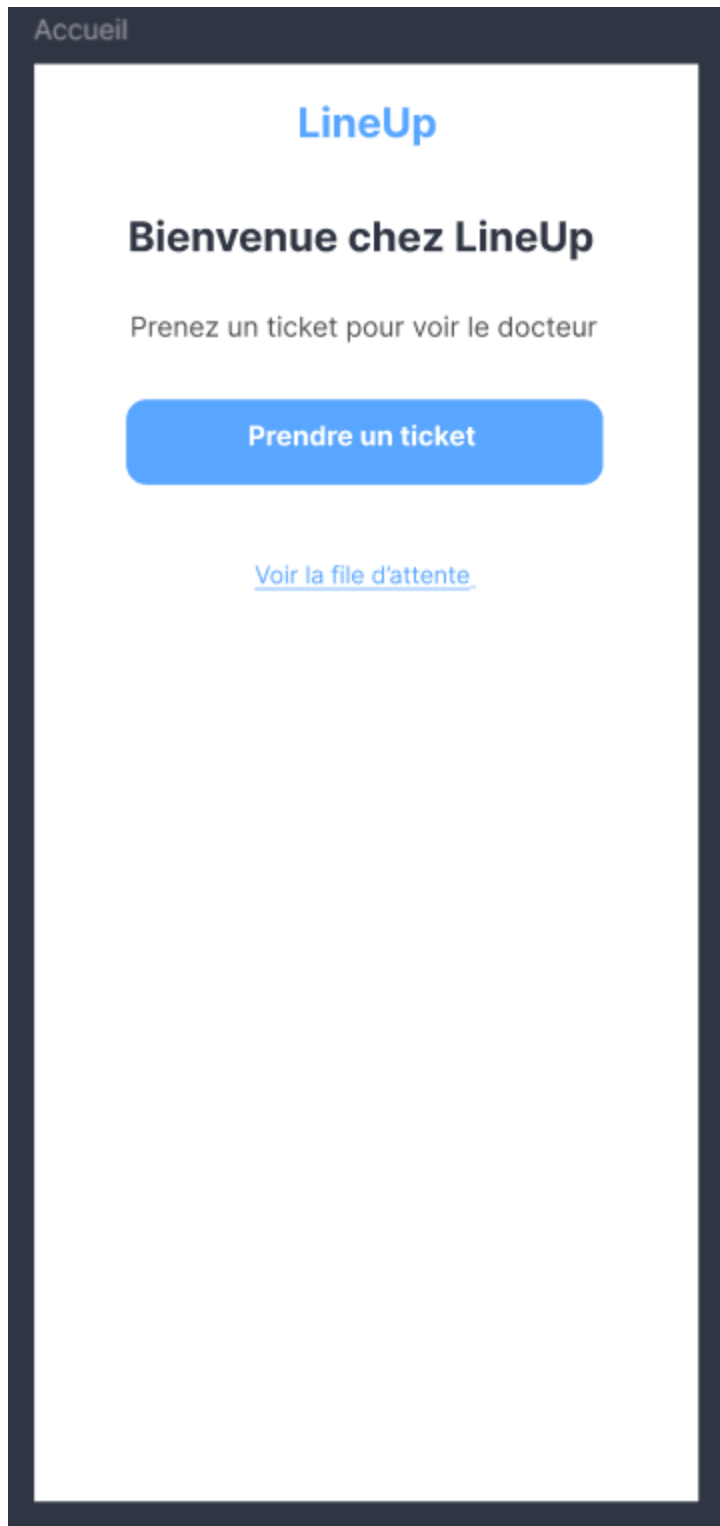
Titre professionnel visé : Développeur Web et Web Mobile (DWWM)

Sommaire

Les maquettes ont été conçues avec Figma en adoptant une approche mobile-first, privilégiant la simplicité, l'accessibilité et la clarté des interactions. Elles illustrent les parcours utilisateurs clés et les principales fonctionnalités de l'application.

Voici une présentation des principales vues commentées :

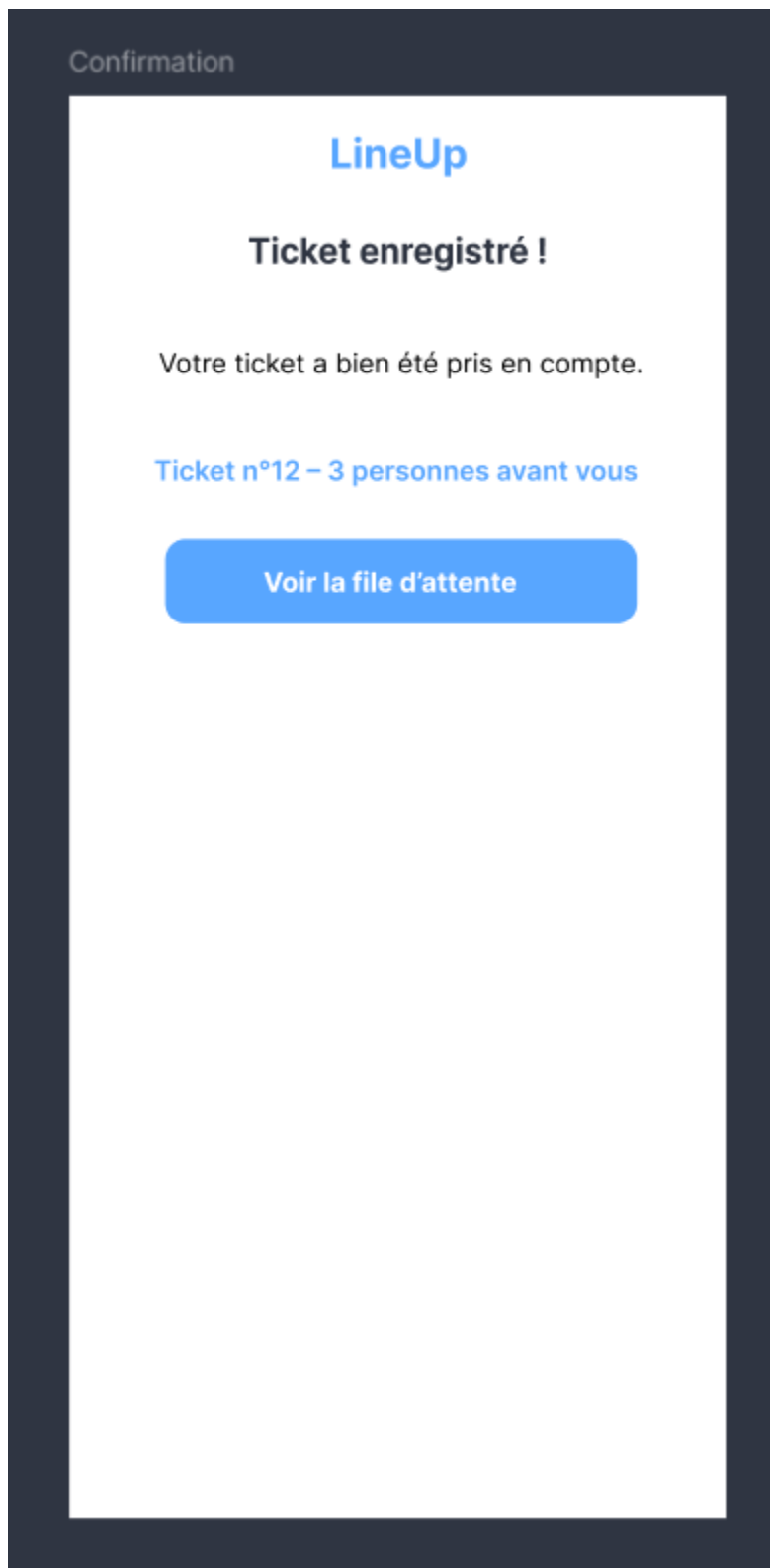
- Écran d'accueil



-

Écran d'accueil Cet écran invite le patient à prendre un ticket. Il présente un bouton CTA clair (« Prendre un ticket ») et un lien secondaire vers la file d'attente. C'est la première interface rencontrée par l'utilisateur.

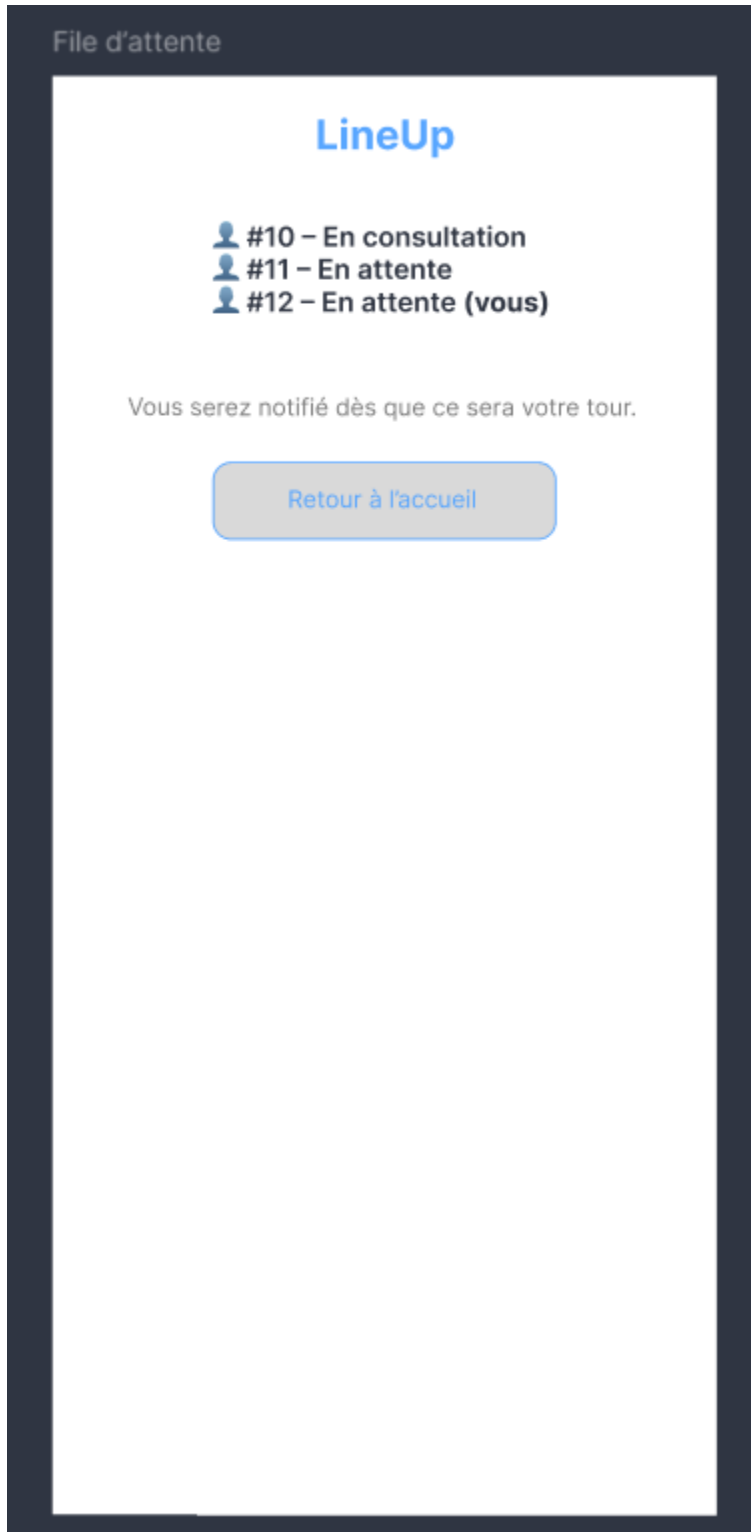
- Confirmation de ticket



-

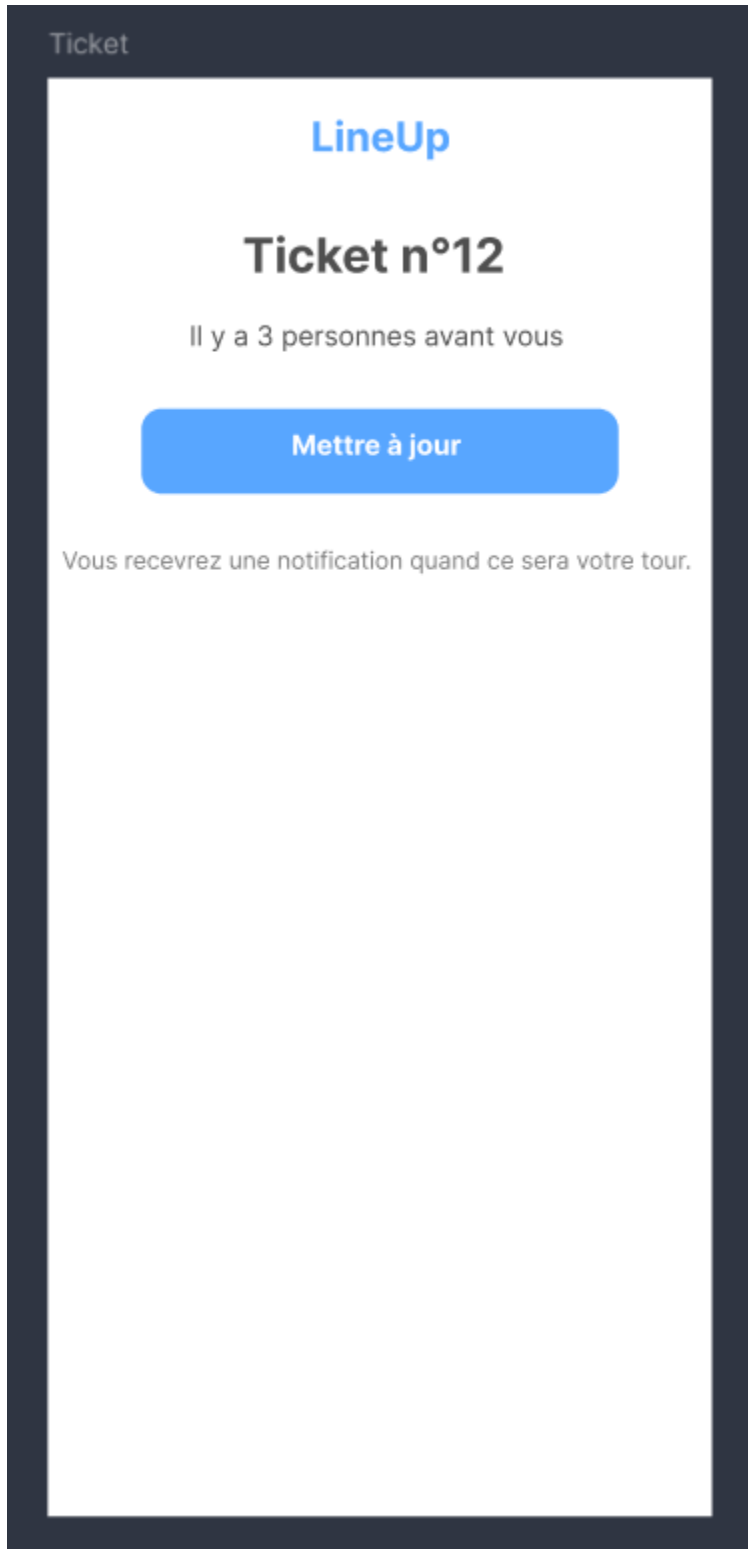
Confirmation de ticket Une fois le ticket validé, cette page informe l'utilisateur de sa position dans la file, avec un lien pour visualiser l'état en temps réel.

- Suivi de la file



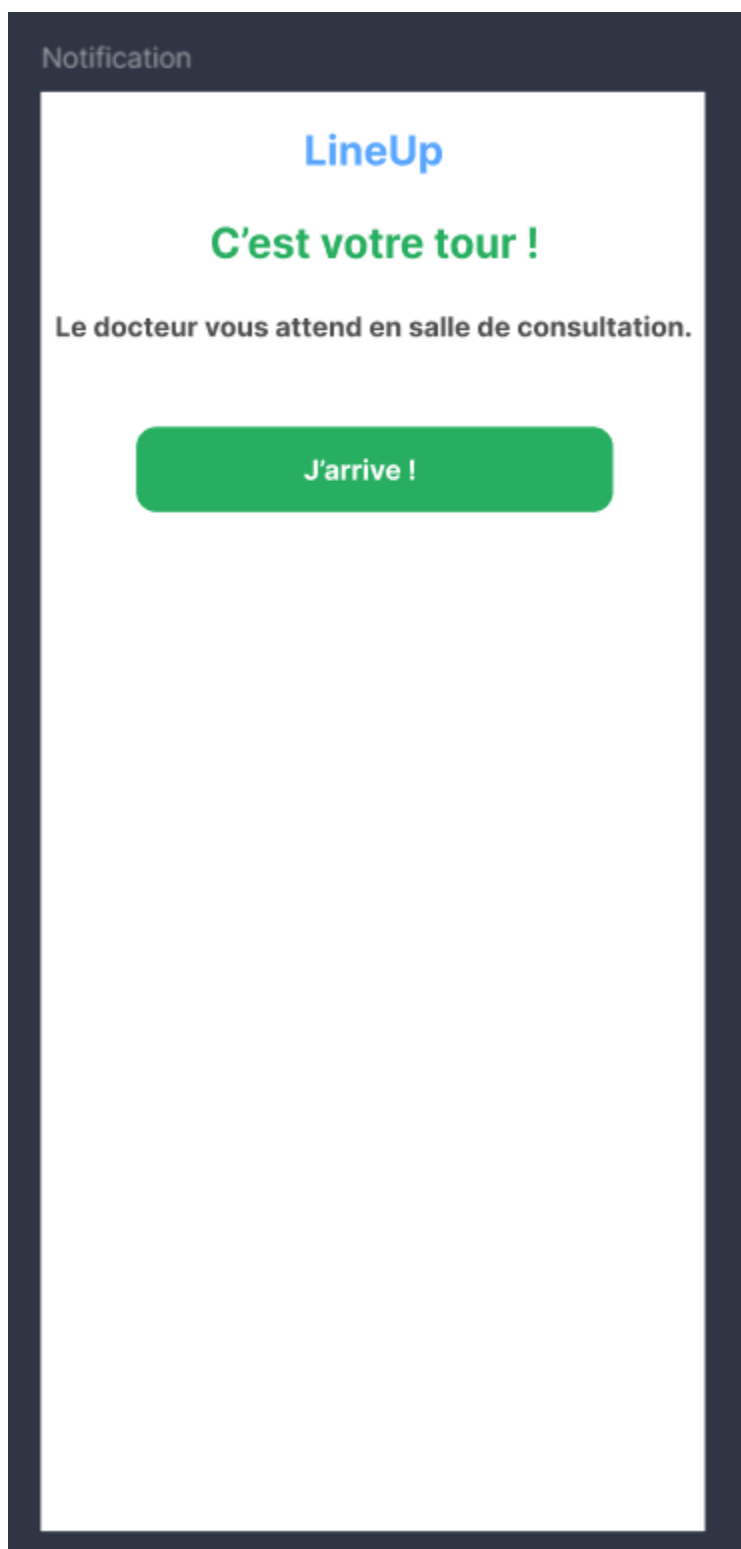
File d'attente La file d'attente est consultable à tout moment. Le statut du patient est mis en évidence pour une meilleure lisibilité.

- Suivi du ticket en temps réel



Suivi du ticket Cette page permet de mettre à jour sa position dans la file, et rappelle le nombre de personnes en attente avant l'utilisateur.

- Notification



-

Notification Lorsqu'un patient est appelé, une notification s'affiche avec un bouton « J'arrive » pour confirmer sa venue immédiate.

- Interface admin



Interface admin Le tableau de bord du médecin ou de la secrétaire affiche les statuts en cours et permet de faire avancer la file via le bouton « Appeler le suivant ».

Ces interfaces renforcent la clarté des parcours utilisateur et garantissent une interaction fluide entre le front et le back-office.

Récapitulatif des maquettes Figma :

- Maquette-Mobile.png → page d'accueil mobile
- Maquette-Desktop.png → page file d'attente version desktop
- Dashboard-Admin.png → tableau de bord médecin/secrétaire
- Interaction-Ticket.png → confirmation et statut d'un ticket
- Ticket-Confirmé-Mobile.png → ticket pris avec QR code
- File-Attente-Statuts.png → affichage des différents statuts dans la file

Chaque image est utilisée pour illustrer des interactions précises. Elles sont placées au cœur du projet pour démontrer la cohérence graphique et fonctionnelle entre conception et réalisation technique.

6. Architecture technique (MERN) L'architecture de LineUp repose sur une stack MERN (MongoDB, Express.js, React, Node.js) moderne, maintenable et évolutive. Elle permet de gérer un flux de données en temps réel tout en garantissant une sécurité des accès grâce au modèle MVC et à une séparation claire des responsabilités.

Structure générale : - □ Frontend : React 18 avec Vite pour le bundling rapide, Tailwind CSS pour un design système cohérent. - □ Backend : Express.js sur Node.js, avec routes RESTful, middlewares pour la sécurité, et logique métier. - □ Base de données : MongoDB Atlas (hébergement cloud sécurisé), avec schémas Mongoose typés. - □ Déploiement : Netlify pour le frontend, Render pour le backend (API REST).

Schéma d'architecture simplifié :

1. Le patient interagit avec le frontend React, qui envoie des requêtes HTTP sécurisées au backend via fetch ou axios.
2. Le backend Express traite les requêtes, applique les middlewares de vérification (authentification JWT, validation de données).
3. Les modèles Mongoose (MongoDB) gèrent les entrées/sorties dans la base de données (tickets, utilisateurs, rôles).
4. La réponse (ticket en file, statut, confirmation, notification) est renvoyée vers le frontend pour affichage en temps réel.

Cette architecture garantit : - Des temps de réponse rapides grâce à Vite et à l'hébergement CDN - Un découplage propre des responsabilités (frontend / backend / DB) - Une sécurité renforcée avec JWT et bcrypt - Une évolutivité possible vers une PWA ou une version mobile avec React Native


```

// Configuration CORS
const allowedOrigins = [
  'https://ligneup.netlify.app',
  'https://lineup.netlify.app',
  'https://lineup-app.netlify.app',
  'http://localhost:5173',
  'http://localhost:5174',
  'http://localhost:3000',
  'https://lineup-backend-xxak.onrender.com'
];

app.use(cors({
  origin: function(origin, callback) {
    // En développement, accepter toutes les origines
    if (process.env.NODE_ENV !== 'production') {
      return callback(null, true);
    }

    // En production, vérifier les origines
    if (!origin || allowedOrigins.includes(origin) || origin.endsWith('.netlify.app'))
  {
    callback(null, true);
  } else {
    console.log('✗ Origine refusée:', origin);
    callback(null, true); // Accepter quand même pour déboguer
  }
},
  credentials: true,
  methods: ['GET', 'POST', 'PUT', 'PATCH', 'DELETE', 'OPTIONS'],
  allowedHeaders: ['Content-Type', 'Authorization', 'X-Requested-With'],
  exposedHeaders: ['Content-Range', 'X-Content-Range'],
  maxAge: 600 // Cache les résultats du pre-flight pendant 10 minutes
}));

```

: server/index.js – Fichier racine du backend : connexion au serveur Express, intégration des routes API]

L'organisation du code suit le modèle MVC : - /controllers → logique métier (ex. gérer les tickets) - /routes → définition des points d'entrée (ex. /api/tickets) - /models → schémas Mongoose (ex. Ticket.js, User.js) - /middlewares → vérifications JWT, rôles, erreurs

Cette architecture a été pensée pour être facilement testable, maintenable, et déployable dans un environnement professionnel. 7. Authentification, rôles et sécurité JWT La sécurité des données personnelles des utilisateurs et la bonne gestion des rôles sont des éléments cruciaux de LineUp. Pour répondre à ces enjeux, un système d'authentification robuste a été mis en place à l'aide de JSON Web Tokens (JWT).

Fonctionnement de l'authentification : - Lorsqu'un utilisateur (admin ou patient) se connecte ou s'inscrit, un token est généré via le module jsonwebtoken. - Ce token est signé avec une clé secrète et contient les données minimales (id, rôle, expiration). - Le token est ensuite stocké côté client (dans le localStorage ou via un cookie sécurisé).

Vérification du token : - À chaque requête nécessitant une authentification (ex : accéder au tableau admin, créer un ticket), un middleware backend vérifie la présence et la validité du token. - En cas de token invalide ou expiré, une erreur est retournée avec un statut 401 (Unauthorized).

Sécurité renforcée : - Utilisation de bcryptjs pour le hachage des mots de passe (aucun mot de passe n'est stocké en clair). - Protection des routes selon les rôles : - Le rôle admin peut consulter la file, appeler les patients, consulter les stats. - Le rôle patient est limité à la création et au suivi de son propre ticket.

Structure technique : - authController.js : gère l'inscription, la connexion et la création de tokens - jwtUtils.js : fonctions de création et vérification des JWT - authMiddleware.js : protège les routes sensibles

```
const generateToken = (payload, secret, options = {}) => {
  try {
    const defaultOptions = {
      expiresIn: '24h',
      issuer: 'lineup-app',
      ...options
    };

    return jwt.sign(payload, secret, defaultOptions);
  } catch (error) {
    console.error('✖ Erreur génération JWT:', error);
    throw new Error('Erreur génération token');
  }
};
```

: jwtUtils.js – Génération du token avec une durée de validité limitée

```

// Middleware pour vérifier l'authentification (obligatoire)
const authenticateRequired = async (req, res, next) => {
  try {
    const token = extractTokenFromHeaders(req.headers);

    if (!token) {
      return res.status(401).json({
        success: false,
        message: 'Token d\'authentification requis'
      });
    }

    const jwtSecret = process.env.JWT_SECRET ||
      'fallbacksecretchangeitbeforeproduction';
    console.warn('⚠ JWT_SECRET manquant - Utilisation du fallback');

    const decoded = verifyToken(token, jwtSecret);

    // Récupérer l'utilisateur complet avec son rôle
    const user = await User.findById(decoded.userId).populate('role');
    if (!user || !user.isActive) {
      return res.status(401).json({
        success: false,
        message: 'Utilisateur non trouvé ou inactif'
      });
    }

    req.user = user;
    req.token = token;
    req.tokenPayload = decoded;
    next();
  } catch (error) {
    console.error('❌ Erreur authentification:', error);
    return res.status(401).json({
      success: false,
      message: error.message || 'Token invalide'
    });
  }
};

```

authMiddleware.js – Vérification du token et extraction de l'identifiant utilisateur

Ce système permet à l'application d'assurer une navigation fluide et sécurisée, tout en protégeant l'intégrité des données médicales et des identités utilisateurs.

8. Base de données (MCD / MLD / MPD)

LineUp utilise une base de données MongoDB pour stocker les informations relatives aux utilisateurs, tickets et statuts. La modélisation de la base a été réalisée selon les étapes classiques :

Modèle Conceptuel de Données (MCD) : - Représente les entités principales : Utilisateur, Ticket, Médecin - Définit les relations : un utilisateur peut avoir un ticket, un médecin gère plusieurs tickets

Modèle Logique de Données (MLD) : - Conversion des entités et relations en collections MongoDB : - Users (patients, médecins, secrétaires) - Tickets (file d'attente, statut, date, ID utilisateur) - Attributs typés et contraintes (email unique, rôle obligatoire, timestamp)

Modèle Physique de Données (MPD) : - Implémenté avec Mongoose (ODM pour MongoDB) - Schémas clairement définis pour assurer la validation des données côté serveur

Objectifs de cette structure : - Assurer la cohérence des données (pas de ticket sans utilisateur) - Rendre la recherche de tickets rapide par statut ou ID - Offrir une scalabilité horizontale si le service grandit

ROLE		
string	name	visiteur, patient, secretaire, medecin
string	displayName	
array	permissions	
string	description	
boolean	isActive	
datetime	createdAt	
datetime	updatedAt	



possède

USER		
string	email	unique
string	password	
object	profile	firstName, lastName, phone, dateOfBirth
object	address	street, city, postalCode, country
object	preferences	notifications, language
boolean	isActive	
datetime	lastLogin	
number	loginCount	
object	pushSubscription	
datetime	createdAt	
datetime	updatedAt	



crée

TICKET		
number	number	unique
string	status	en_attente, en_consultation, desiste, termine
string	docteur	dr-husni, dr-helios, dr-jean-eric
string	patientName	pour tickets physiques
string	ticketType	numerique, physique
string	createdBy	patient, secretary, system
string	notes	
object	metadata	ipAddress, userAgent, device
datetime	createdAt	
datetime	updatedAt	



consulte

DOCTOR		
string	id	dr-husni-said-habibi, dr-helios-blasco, dr-jean-eric-panacciulli
string	firstName	
string	lastName	
string	email	
string	phone	
string	specialite	

peut être

PATIENT		
string	email	unique
string	password	

: MCD – Vue des entités principales et leurs relations logiques

roles			
ObjectId	_id	PK	
string	name		UNIQUE, ENUM
string	displayName		
array	permissions		
string	description		
boolean	isActive		DEFAULT true
datetime	createdAt		
datetime	updatedAt		

patients			
ObjectId	_id	PK	
string	email		UNIQUE, REQUIRED
string	password		REQUIRED

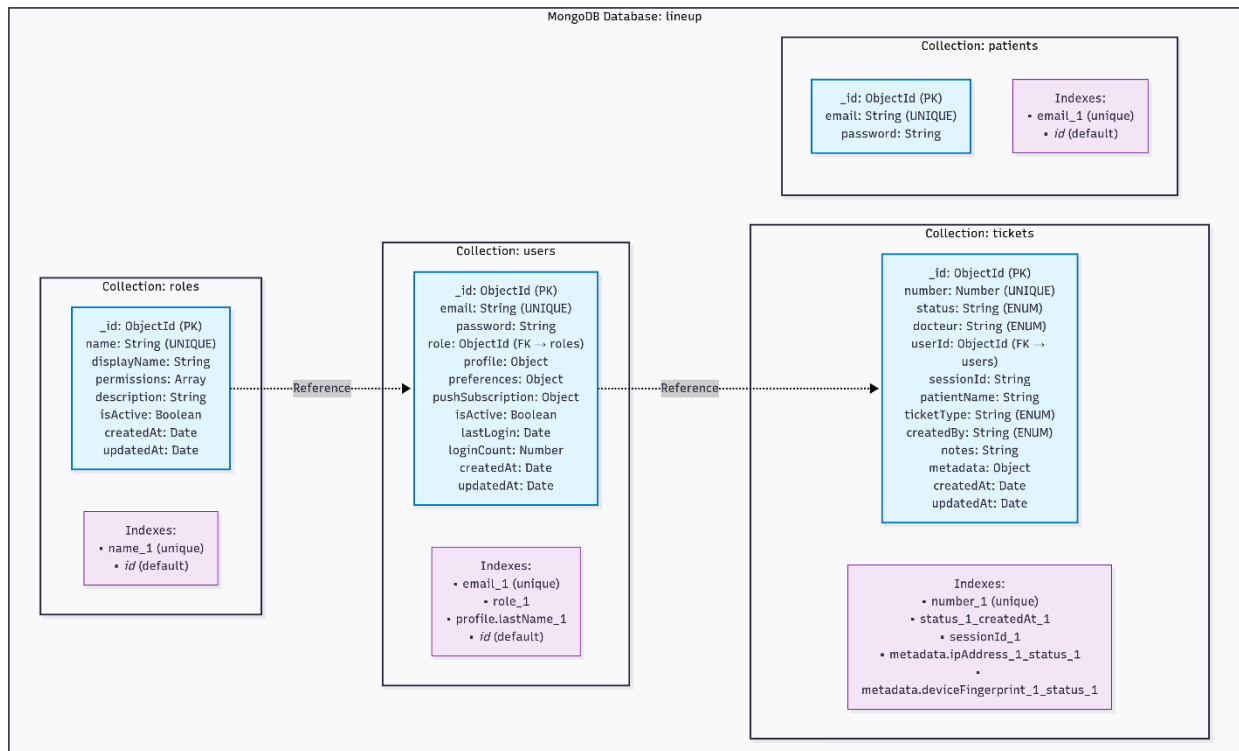
⌞
role_id

users			
ObjectId	_id	PK	
string	email		UNIQUE, REQUIRED
string	password		REQUIRED
ObjectId	role	FK	REQUIRED
string	profile_firstName		
string	profile_lastName		
string	profile_phone		
date	profile_dateOfBirth		
string	address_street		
string	address_city		
string	address_postalCode		
string	address_country		DEFAULT France
boolean	preferences_notifications_email		DEFAULT true
boolean	preferences_notifications_sms		DEFAULT false
string	preferences_language		DEFAULT fr
boolean	isActive		DEFAULT true
datetime	lastLogin		
number	loginCount		DEFAULT 0
string	pushSubscription_endpoint		
string	pushSubscription_keys_p256dh		
string	pushSubscription_keys_auth		
datetime	createdAt		
datetime	updatedAt		

⌞
userId

tickets			
ObjectId	_id	PK	
number	number		UNIQUE, REQUIRED
string	status		ENUM, DEFAULT en_attente
string	docteur		ENUM, REQUIRED
ObjectId	userId	FK	NULLABLE
string	sessionId		NULLABLE
string	patientName		NULLABLE, MAX 100
string	ticketType		ENUM, DEFAULT numerique
string	createdBy		ENUM, DEFAULT patient
string	notes		NULLABLE, MAX 500
string	metadata_ipAddress		
string	metadata_userAgent		
string	metadata_device		
string	metadata_deviceFingerprint		
datetime	createdAt		
datetime	updatedAt		

:MLD – Définition des collections et des types MongoDB



:MPD – Schémas Mongoose implémentés dans le projet

Grâce à cette modélisation rigoureuse, l'application bénéficie d'une base fiable, évolutive et cohérente avec le fonctionnement global de la file d'attente. 9. Frontend (React, composants, hooks) Le frontend de LineUp a été conçu pour offrir une expérience fluide, responsive et moderne. Il repose sur React 18, Vite pour un bundling rapide, et Tailwind CSS pour une personnalisation visuelle efficace. L'approche mobile-first a été strictement respectée.

- Architecture du frontend : - App.jsx : point d'entrée principal de l'application React - Routes.js : définition des routes utilisateurs/patients/admin avec React Router - Components/ : composants UI réutilisables (QueueCard, StatusBadge, Button) - Hooks/ : hooks personnalisés pour le rafraîchissement en temps réel, l'état global, ou les appels API (useRealTimeQueue)
- Fonctionnalités clés : - Prise de ticket depuis la page d'accueil - Suivi en direct de sa position dans la file - Réception d'une notification visuelle + sonore lorsqu'on est appelé - Vue responsive adaptée au mobile et desktop
- Réactivité en temps réel : Le hook useRealTimeQueue interroge régulièrement le backend pour mettre à jour la file d'attente en direct, sans avoir besoin de recharger la page. Cela améliore fortement l'expérience utilisateur.


```

{ /* Navigation desktop - FORCÉ INVISIBLE sur mobile */ }
<div className="hidden lg:flex space-x-6 force-hide-mobile">
  <Link
    to="/"
    className="flex items-center space-x-1 px-3 py-2 rounded-lg text-gray-700 hover:text-blue-600
    hover:bg-blue-50 transition-all duration-200"
  >
    <span>🏠</span>
    <span className="font-medium">Accueil</span>
  </Link>
  <Link
    to="/queue"
    className="flex items-center space-x-1 px-3 py-2 rounded-lg text-gray-700 hover:text-blue-600
    hover:bg-blue-50 transition-all duration-200"
  >
    <span>📄</span>
    <span className="font-medium">File d'attente</span>
  </Link>
  { !isAuthenticated && (
    <Link
      to="/ticket"
      className="flex items-center space-x-1 px-3 py-2 rounded-lg text-gray-700 hover:text-blue-600
      hover:bg-blue-50 transition-all duration-200"
    >
      <span>🎫</span>
      <span className="font-medium">Mon ticket</span>
    </Link>
  ) }
</div>

```

: App.jsx – Point d'entrée React, intégration des routes et structure des layouts

```
const getStatusConfig = (status) => {
  switch (status) {
    case 'en_attente':
      return {
        icon: '🕒',
        label: 'En attente',
        bgClass: 'bg-blue-50 border-blue-200',
        badgeClass: 'bg-blue-100 text-blue-800',
        textClass: 'text-blue-900'
      };
    case 'en_consultation':
      return {
        icon: '👨‍⚕️',
        label: 'En consultation',
        bgClass: 'bg-green-50 border-green-200',
        badgeClass: 'bg-green-100 text-green-800',
        textClass: 'text-green-900'
      };
    case 'termine':
      return {
        icon: '✅',
        label: 'Terminé',
        bgClass: 'bg-gray-50 border-gray-200',
        badgeClass: 'bg-gray-100 text-gray-700',
        textClass: 'text-gray-700'
      };
    case 'desiste':
      return {
        icon: '❌',
        label: 'Annulé',
        bgClass: 'bg-red-50 border-red-200',
        badgeClass: 'bg-red-100 text-red-800',
        textClass: 'text-red-900'
      };
    default:
      return {
        icon: '❓',
        label: status,
        bgClass: 'bg-gray-50 border-gray-200',
        badgeClass: 'bg-gray-100 text-gray-700',
        textClass: 'text-gray-700'
      };
  }
};
```

:ImprovedQueueCard.jsx – Affichage du ticket avec statut dynamique et bouton d'action

```
// Hook personnalisé pour gérer le cache des files d'attente
const useQueueCache = (selectedDoctor) => {
  const cacheKey = selectedDoctor || 'all';

  const getCacheData = useCallback(() => {
    cleanQueueCache();
    const cached = QUEUE_CACHE.get(cacheKey);
    if (cached && Date.now() - cached.timestamp < CACHE_DURATION) {
      console.log(`📁 Données de file en cache trouvées pour ${cacheKey}`);
      return cached.data;
    }
    return null;
  }, [cacheKey]);

  const setCacheData = useCallback((data) => {
    cleanQueueCache();
    QUEUE_CACHE.set(cacheKey, {
      data,
      timestamp: Date.now()
    });
    console.log(`📁 File d'attente mise en cache pour ${cacheKey}`);
  }, [cacheKey]);

  return { getCacheData, setCacheData };
};
```

```

// Fonction de fetch avec gestion d'erreur améliorée et cache
const fetchQueue = useCallback(async (useCache = true) => {
  if (!isActiveRef.current || !isMountedRef.current) return;

  // Annuler toute requête en cours
  if (abortControllerRef.current) {
    abortControllerRef.current.abort();
  }

  // Créer un nouveau controller pour cette requête
  abortControllerRef.current = new AbortController();
  const signal = abortControllerRef.current.signal;

  // Vérifier le cache en premier si demandé
  if (useCache && !isLoading) {
    const cachedData = getCacheData();
    if (cachedData) {
      console.log(` ⚡ Utilisation du cache pour la file: ${selectedDoctor ||
'all'} $$$$ Queue(cachedData.queue);
      setLastUpdate(new Date(cachedData.lastUpdate).getTime());
      setError(null);
      setIsConnected(true);
      setIsTransitioning(false);
      return;
    }
  }
}

```

:useRealTimeQueue.js – Hook personnalisé interrogeant l'API toutes les X secondes

Cette architecture frontend garantit une application performante, claire, et orientée utilisateur. 10. Backend (Node, Express, routes API) Le backend de LineUp est construit avec Node.js et le framework Express.js. Il repose sur une architecture MVC claire pour séparer la logique métier, les routes et la manipulation des données via Mongoose.

Structure générale : - /controllers : logique métier (gestion des tickets, authentification, notifications) - /routes : définition des points d'accès à l'API RESTful (ex : /api/tickets, /api/users) - /models : schémas de données Mongoose pour Users, Tickets - /middlewares : sécurisation des routes avec authMiddleware, gestion des rôles, gestion des erreurs

Fonctionnalités du backend : - Authentification avec JWT (login/register) - Sécurisation des routes via middleware (admin, patient) - Création et suivi de tickets (GET, POST, PATCH, DELETE) - Appel du patient suivant par l'admin - Notification et changement de statut (en attente → en consultation → terminé)

Communication frontend/backend : Le backend expose une API REST consommée par le frontend React. Chaque appel à l'API est vérifié via un token JWT, puis redirigé vers le bon contrôleur.

□ Exemple de cycle : 1. Le patient clique sur « Prendre un ticket » → appel POST /api/tickets 2. Le middleware vérifie le token → passage au ticketController 3. Le ticket est enregistré en base, un statut « en attente » est attribué 4. Le frontend reçoit une réponse JSON confirmant l'action

```
// Middleware pour vérifier l'authentification (obligatoire)
const authenticateRequired = async (req, res, next) => {
  try {
    const token = extractTokenFromHeaders(req.headers);

    if (!token) {
      return res.status(401).json({
        success: false,
        message: 'Token d\'authentification requis'
      });
    }

    const jwtSecret = process.env.JWT_SECRET ||
      'fallbackSecretChangeIfPossible';
    console.warn('⚠ JWT_SECRET manquant - Utilisation du fallback');

    const decoded = verifyToken(token, jwtSecret);

    // Récupérer l'utilisateur complet avec son rôle
    const user = await User.findById(decoded.userId).populate('role');
    if (!user || !user.isActive) {
      return res.status(401).json({
        success: false,
        message: 'Utilisateur non trouvé ou inactif'
      });
    }

    req.user = user;
    req.token = token;
    req.tokenPayload = decoded;
    next();
  } catch (error) {
    console.error('❌ Erreur authentification:', error);
    return res.status(401).json({
      success: false,
      message: error.message || 'Token invalide'
    });
  }
};
```

: authMiddleware.js – Middleware de vérification des tokens

```

router.post('/login', async (req, res) => {
  try {
    const { email, password } = req.body;

    // Validation des champs
    if (!email || !password) {
      return res.status(400).json({
        message: 'Email et mot de passe requis'
      });
    }

    // Trouver l'utilisateur avec son rôle
    const user = await User.findOne({
      email: email.toLowerCase(),
      isActive: true
    }).populate('role');

    if (!user) {
      return res.status(401).json({
        message: 'Email ou mot de passe incorrect'
      });
    }

    // Vérifier le mot de passe
    const isPasswordValid = await bcrypt.compare(password, user.password);
    if (!isPasswordValid) {
      return res.status(401).json({
        message: 'Email ou mot de passe incorrect'
      });
    }

    // Générer le JWT avec notre utilitaire robuste
    const jwtSecret = process.env.JWT_SECRET ||
    'fallback_secret_change_it_in_production';
    {
      userId: user._id,
      email: user.email,
      role: user.role.name,
      permissions: user.role.permissions
    },
    jwtSecret,
    { expiresIn: '24h' }
  );
}

```

:auth.js – Routes d'authentification et gestion des rôles

```

// Créer un nouveau ticket
exports.createTicket = async (req, res) => {
  try {
    const { docteur } = req.body;

    if (!docteur) {
      return res.status(400).json({ message: "Le docteur est requis" });
    }

    const number = await generateTicketNumber(docteur);

    const ticket = new Ticket({
      number,
      docteur,
      status: 'en_attente',
      metadata: {
        ipAddress: req.ip,
        userAgent: req.headers['user-agent'],
        device: req.headers['user-agent']
      }
    });

    if (req.user) {
      ticket.userId = req.user._id;
    } else {
      ticket.sessionId = req.sessionID;
    }

    await ticket.save();
    res.status(201).json(ticket);
  } catch (error) {
    console.error('Erreur création ticket:', error);
    res.status(500).json({ message: "Erreur lors de la création du ticket" });
  }
};

```

:ticketController.js – Contrôleur principal de gestion des tickets


```
const userSchema = new mongoose.Schema({
  email: {
    type: String,
    required: true,
    unique: true,
    lowercase: true,
    trim: true
  },
  password: {
    type: String,
    required: true
  },
  role: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'Role',
    required: true
  },
  profile: {
    firstName: String,
    lastName: String,
    phone: String,
    dateOfBirth: Date,
    address: {
      street: String,
      city: String,
      postalCode: String,
      country: { type: String, default: 'France' }
    }
  },
  preferences: {
    notifications: {
      email: { type: Boolean, default: true },
      sms: { type: Boolean, default: false }
    },
    language: { type: String, default: 'fr' }
  },
  isActive: {
    type: Boolean,
    default: true
  },
  lastLogin: {
    type: Date,
    default: null
  },
  loginCount: {
    type: Number,
    default: 0
  },
  pushSubscription: {
    endpoint: String,
    keys: {
      p256dh: String,
      auth: String
    }
  }
}, {
  timestamps: true
});
```

:User.js – Schéma utilisateur avec validation (Mongoose)

Cette organisation permet de maintenir un code clair, modulaire et extensible. Le backend est pensé pour être facilement testé et adapté à des évolutions futures (ex : ajout de statistiques, gestion des médecins, notifications par mail). 11. Tests, déploiement et maintenance

Tests fonctionnels CRUD avec HTTPie : Chaque fonctionnalité clé de l'API (authentification, création de tickets, mise à jour de statut, suppression) a été testée avec HTTPie, en simulant les appels front → back en conditions réelles. Les cas suivants ont été couverts : - Création de ticket avec token valide - Erreur 401 si token manquant ou invalide - Mise à jour du statut (admin uniquement) - Suppression d'un ticket terminé

Stratégie de déploiement : - Le frontend est déployé sur Netlify à l'adresse <https://ligneup.netlify.app> - Le backend est hébergé sur Render (service Node.js gratuit) avec endpoints REST exposés publiquement

CI/CD (mise à jour continue) : - Les changements du code front ou back sont automatiquement redéployés via les hooks Git dès qu'un commit est effectué sur les branches principales

Maintenance : - L'API est pensée pour être facilement étendue (ajout de fonctionnalités médicales ou statistiques) - Le code est commenté, modulaire, et utilise des conventions claires (nommage, separation of concerns, fichiers centralisés de configuration)

Résultat : L'application est stable, rapide, et son déploiement est facilement reproductible. Des tests manuels réguliers ont permis de garantir le bon fonctionnement sur différents supports (mobile, desktop) et navigateurs. 12. Annexes et liens de production\$1

Fiche de compétences : Mise en œuvre du projet LineUp

Ce projet couvre les 8 compétences professionnelles du titre professionnel DWWM :

CP1 – Installer et configurer son environnement de travail

- ☐ Installation de Node.js, MongoDB, Vite, React, Postman, HTTPie
- ☐ Configuration des variables d'environnement (.env) pour les clés JWT et les URI MongoDB
- ☐ Utilisation de Git, VS Code, Render, Netlify et MongoDB Atlas

CP2 – Maquetter des interfaces utilisateur web ou web mobile

- ☐ Réalisation de maquettes Figma mobile-first et desktop
- ☐ Conception de parcours utilisateurs et UI responsive
- ☐ Respect d'une charte graphique cohérente et accessible

CP3 – Réaliser des interfaces utilisateur statiques web ou web mobile

- ☐ Intégration en React + Tailwind CSS
- ☐ Responsive design et structuration HTML sémantique
- ☐ Accessibilité et compatibilité mobile/desktop

CP4 – Développer la partie dynamique des interfaces utilisateur web ou web mobile

- ☐ Utilisation de React Hooks, Context, Router pour la dynamique
- ☐ Gestion de l'état, des rôles, des redirections et de la file en temps réel
- ☐ Intégration d'une API sécurisée avec axios et gestion des erreurs

CP5 – Mettre en place une base de données relationnelle

- ☐ Conception du MCD/MLD avec MongoDB comme base NoSQL
- ☐ Création de collections utilisateurs, tickets et rôles
- ☐ Utilisation de MongoDB Atlas pour l'hébergement et le monitoring

CP6 – Développer des composants d'accès aux données SQL et NoSQL

- ☐ Création des modèles Mongoose avec contraintes
- ☐ Requêtes CRUD sécurisées sur les tickets, utilisateurs, sessions
- ☐ Validation des entrées et gestion des erreurs serveur

CP7 – Développer des composants métier côté serveur

- ☐ Définition des routes sécurisées (middleware auth, rôles)
- ☐ Contrôleurs métiers (prise de ticket, statut, notification)
- ☐ Système de file d'attente temps réel par statut patient

CP8 – Documenter le déploiement d'une application dynamique web ou web mobile

- ☐ Déploiement Netlify (front) et Render (back)
- ☐ Explication de la structure, environnement distant, CI/CD Git
- ☐ Documentation claire des endpoints API testés via HTTPie

pour optimiser la gestion des files d'attente médicales, améliorant significativement le quotidien des établissements et la satisfaction des patients.

Conclusion

LineUp est bien plus qu'un simple projet d'école : c'est une solution complète pensée pour répondre à une problématique réelle et fréquente dans les salles d'attente. Il

démontre une maîtrise complète de la stack MERN, des enjeux UX/UI, de la sécurité web, du déploiement professionnel, et de la documentation.

Ce projet m'a permis de mettre en œuvre l'ensemble des compétences attendues pour le titre de Développeur Web et Web Mobile, et m'a sensibilisé aux exigences concrètes d'un projet en conditions réelles.

Merci à l'équipe pédagogique, aux testeurs, et à toutes les personnes qui m'ont soutenu dans ce travail.
